

API功能业务测试用例方案

1. 业务测试用例设计表

模块	接口	业务测试用例设计	测试结果
repo	创建 比对 库	1. 针对replications参数中设定只能向上修改功能，设计如下测试用例： a. 创建比对库testrepo-replications，设置replications=1，expect success b. 修改比对库testrepo-replications，设置replications=3，expect success c. 查询比对库testrepo-replications，检查replications=3，验证b是否成功修改，expect success d. 再次修改比对库testrepo-replications，设置replications=3，expect false e. 查询比对库testrepo-replications，检查replications=3，验证d是否修改失败，expect success f. 删除比对库testrepo-replications，expect success	1. Test error: replications字段无法调用update接口修改，步骤b无法执行，无法正常进行测试 a. 001_style_api/业务测试/业务测试脚本/repo/replications.py 步骤b返回 <pre>1 { 2 "method": "PATCH", 3 "path": 4 "/repositories/testrepo- 5 replications", 6 "status_code": 501, 7 "response": { 8 "error": 9 "updateRepository is 10 not implemented yet", 11 "error_code": 12 "9000001" 13 } 14 }</pre>
		1. 针对capacity库大小达到或超过库容，新入库请求会被拒绝功能，设计如下测试用例： a. 删除比对库 testrepo-capacity（如果存在） b. 创建比对库 testrepo-capacity，容量为 7	2. Test success: 针对capacity库大小达到或超过库容，新入库请求会被拒绝功能 a. 001_style_api/业务测试/业务测试脚本/repo/capacity.py 第八个entity返回 <pre>1 { 2 "method": "POST",</pre>

	c. 向比对库中添加 7 个实体，验证添加成功 d. 尝试添加第 8 个实体，验证添加失败	<pre> 3 "path": "/repositories/testrepo- capacity/entities", 4 "status_code": 500, 5 "response": { 6 "error": "200006: RepositoryTsOps::addTsF eaturesBatch(testrepo- capacity/testrepo- capacity): capacity: RepositoryCapacityGc::e nforceCapacityAndCleanu p(testrepo- capacity/testrepo- capacity): Repository capacity exceeded: capacity=7, active=7, incoming=1", 7 "error_code": "200006" 8 } 9 }</pre>
	1. 针对rotate功能，设计以下测试用例 a. 删除并重建 testrepo-rotate，确保 rotate 场景从干净环境开始。 b. 创建时显式开启 Rotate，确认仓库配置可回显该开关。 c. 连续新增 5 条实例，作为基础数据。 d. 获取实例entity1，expect success e. 再新增实例entity6 f. 获取原entity1-6，expect get entity1-2 error,其余success	1. Test error: 测试用例当中的entity1不存在entity2与entity6存在，说明repo内只删除最早一条实例，size仍保持capacity=5的大小，与接口文档表述有出入。 a. 补充： i. 插入实例后，repo列表当中更新size大小的时间非常长，无法实时查看插入实例后的repo的size大小 b. 001_style_api/业务测试/repo/rotate.py
	1. 针对PreFilter功能，设计如下方案进行验证。ram模式下有效	1. 测试location_id过滤功能，失败，location_id前过滤也未实现，前面search的合法校验也发现这个问题 a. (Expect output 5 of entity 11-15) (real_output 2 of entity 11-15 and 3 of entity 21-23) recall 40%

- a. 拿到实例1 2 3，基于实例3特征，对存有实例1 2的比对库repo进行search接口调用。假设其中实例1与实例3的相似度为0.90，实例2与实例3相似度为0.99（topk的排序标准是similarity）。
- b. 添加20个实例，（5条实例1设定location_id为1，5条实例1设定location_id为0，时间戳设定为t1（记录第一个和最后一个实例创建时间戳），5条实例2设定location_id为1，5条实例2设定location_id为0,时间戳设定为t2（记录第一个和最后一个实例创建时间戳）、repo开启PreFilter、处理器模式为非npu的比对库repo，基于实例3，结合调用search接口进行验证。（max_candidates--topk设定为5）

c. repo1内实例排布



d. repo2内实例排布。-添加三条entity3



e. 前过滤：先进行time与location_id进行过滤，再进行topk统计

i. 测试环境

1. case1 3 使用repo1测试
2. case2 使用repo2测试

ii. Case design:

2. 其余两条测试用例由于time过滤未实现，测试失败，search返回空值
 - a. `AssertionError: {'message': '', 'results': []}`
3. 001_style_api/业务测试/repo/prefilter.py

- 测试location_id与time过滤功能
设定location_id为1, start_time为t0, end_time为t1, 调用search接口,
(Expect output 5 of entity 1-5)
(real_output expect 5 of entity 1-5)
recall 100%
- 测试location_id过滤功能 添加3条实例3-设定location_id为0, input: 设定location_id为1, 调用search接口, (Expect output 5 of entity 11-15)
(real_output expect 5 of entity 11-15)
recall 100%
- 测试time过滤功能 设定start_time为t0, end_time为t1, 调整max_candidates-topk为10, 调用search接口,
(Expect output 10 of entity 1-10)
(real_output expect 10 of entity 1-10)
recall 100%

f. 经过以上两个样例得知 前过滤功能成功实现 指标为 output_results recall

	1. 针对UseFeatureIDMap功能，调用获取比对库接口确定该功能是否开启（数据存储到内存后如何安全删除） a. 调用添加实例接口，添加500w条实例，查看当前服务器内存占用	5. Test error: 当前版本暂不支持 代码块<pre>1 "error": "options.UseFeatureIDMap=true is not supported in this version", 2 "error_code": "100001"</pre> a. 001_style_api/业务测试/repo/useFeatureIDMap.py
	1. 针对timerDeleteDays相关功能，开启后比对库会根据时间对过期特征进行删除 a. 流程： i. 删除并重建timerDeleteDays 比对库，确保测试从干净环境开始。 ii. 创建时显式开启TimerDeleteDays=1，并等待仓库进入 READY。 iii. 按接口要求使用 body.id 新增 3 条实例，验证写入链路正常。 iv. 查询比对库配置，确认TimerDeleteDays 开关可正常回显。	3. Test success: 第二天成功删除 <pre>{ "capacity": 0, "id": "timerDeleteDays", "index_type": "int8", "level": "ram", "options": { "DcpmmEnabled": "false", "KnnEnabled": "false", "KnnNeighbors": "5", "KnnThreshold": "0.5", "PreFilter": "true", "QEEnabled": "false", "QENeighbors": "5", "QEPow": "3", "QETHreshold": "0.45", "Rotate": "false", "TimerDeleteDays": "1", "UseFeatureIDMap": "false", "hashingNum": "" }, "replications": 1, "size": 3, "status": "READY", "type": "face" }</pre>
	2. TODO_可尝试测试 针对当前库版本会拒绝不同版本的实例的功能，调用获取对比库设置接口确定当前库版本号，调用添加实例接口 -不支持 a. 设定实例版本与当前库版本不同，expect error i. 删除比对库 entity-repo-version-test（如果存在） ii. 创建比对库 entity-repo-version-test，版本为 v1 iii. 向比对库中添加 1 个实体，版本为v1，expect success	1. Test error: 未实现版本控制 <pre>{ "index": 6, "method": "POST", "path": "/repositories/entity-repo-version-test/entities", "status_code": 201, "payload": { "id": "entity-1", "data": { "type": "feature", "value": "CJFOIoAQAGDRPABjr0AgBM8AEA/PQAA57sAwCM9AGC4PA", "version": "v2" }, "location_id": "loc-001" }, "response": { "id": "entity-1" } }</pre>

		iv. 向比对库中添加 1 个实体，版本为v2，expect error ☐ TODO 针对不同库使用不同分数映射版本进行 case design	
		1. TODO hashingNum case design	
获取 比对 库列 表	- 辅助创建和删除比对库接口校验操作	1. case design ☐ 针对创建比对库功能 i. 创建 testrepo 比对库。 ii. 用 get 接口获取该比对库是否存在。 ☐ 针对删除比对库功能， i. 创建testrepo-delete比对库 ii. 调用get接口获取比对库列表，返回testrepo存在 iii. 删除testrepo比对库 iv. 调用delete接口再删除一次，返回删除失败	1. 针对创建比对库功能，Test success a. 001_style_api/业务测试/repo/create_repo.py 2. 针对删除比对库功能，Test success，但删除后get获取库已删除有较大延迟 a. 001_style_api/业务测试/repo/delete_repo.py
修改 比对 库配 置	1. Case design	a. 针对各个合法参数的修改场景 ☐ 调用修改比对库配置接口，修改 relications参数，调用获取比对库配置是否成功修改 ☐ 调用修改比对库配置接口，修改Capacity参数，调用获取比对库配置是否成功修改 ☐ 调用修改比对库配置接口，修改Rotate参数，调用获取比对库配置是否成功修改	1. update未实现 ☐ 调用修改比对库配置接口，修改 relications参数，调用获取比对库配置是否成功修改 返回501code 建议：检查是否成功实现

		☐ 调用修改比对库配置接口，修改TimerDeleteDays参数，调用获取比对库配置是否成功修改 ☐ 调用修改比对库配置接口，修改QEEnabled参数，调用获取比对库配置是否成功修改 ☐ 调用修改比对库配置接口，修改KnnEnabled参数，调用获取比对库配置是否成功修改 b. 以及对合法参数修改后，功能是否按照修改后的配置进行执行场景 ☐ 做创建比对库中的case_situation1验证-replications ☐ 做创建比对库中的case_situation2验证-capacity ☐ 做创建比对库中的case_situation3验证-rotate ☐ 做创建比对库中的case_situation6验证-TimerDeleteDays	
	获取 比对 库配 置	- 辅助上一个修改比对库配置接口进行测试	1. 已完成
	删除 指定 比对 库	1. Case design a. 针对删除比对库功能	1. 已完成

		☐ 调用删除比对库接口，删除比对库id为repo，调用获取比对库列表接口，验证是否删除成功 - 上文已做测试 ☐ 调用删除比对库接口，删除比对库id为repo_invaild，调用获取比对库列表接口，expect error - 已做非法测试	
entity	添加实例	- 针对调用compare接口所有entity版本需要一致需求，设计如下用例 ☐ 设定两个entity为feature type，value为特征，版本相同，调用compare接口，expect success ☐ 设定两个entity为feature type，value为特征，版本不同，调用compare接口，expect error ☐ 设定比对库和实例版本一致，添加，expect success ☐ 设定比对库和实例版本不一致，添加，expect error - 针对重复repo_id添加情况 ☐ 添加重复entity_id expect error - 针对添加entity当中的value与type兼容问题 -非法测试已完成	- 针对调用compare接口所有entity版本需要一致需求 - 测试用例1成功，测试用例2失败，返回success，未实现版本控制 - 测试用例3成功，测试用例4失败，返回success，未实现版本控制 001_style_api/业务测试/entity/compare_version.py - 针对重复repo_id添加情况,测试通过，code409 001_style_api/业务测试/entity/readd_entity.py 代码块 <pre> 1 "error": "400011: PersistenceManager::add FeaturesBatchDetailed(float32)(n=1): MongoDBClient::insertFeaturesBatchDetailed(n=1): E11000 duplicate key error collection: vecdb_new_02.features index: repo_id_1_index_name_1_ external_id_1 dup key: { repo_id: \"testrepo- readd\", index_name: \"testrepo-readd\", external_id: \"rotate- 1\" }\", </pre>

			2 "error_code": "400011"
	获取实例内容	1. 针对获取实例内容功能。-与合法测试用例重合 ☐ 添加一个实例，再获取该实例内容，看是否一致 ☐ 删除一个实例，再获取该实例内容，expect error	1. 已完成
	修改实例内容	1. 针对request body可为空，可自动读取url中的entity_id填充功能 ☐ 设定id为空，调用修改比对库实例接口 (expect response body中的id为url当中的entity_id) 2. 针对各个合法参数的修改场景以及对合法参数修改后，功能是否按照修改后的配置进行执行场景 ☐ 调用修改实例内容接口，修改value与type参数，调用获取比对库配置是否成功修改，再做search的case_situation1验证 ☐ 调用修改实例内容接口，修改version参数，调用获取比对库配置是否成功修改，再做添加实例的case_situation1验证	1. update未实现 ☐ 调用修改实例内容接口，修改参数，调用获取比对库配置是否成功修改 响应code501 查看是否成功实现
	删除实例	1. 针对单机版删除不存在entity_id时返回500及错误信息功能。-与非空测试用例重合 ☐ 设定一个不存在的entity_id，调用删除实例接口，expect error 500	1. 已完成
search	搜索	1. TODO - 针对type为feature_id以及value为feature_id时repo_id生效的场景测试 a. 目前有entity1 entity2 entity3 repo1 repo2	1. TODO 未实现，优先级低 先skip 2. 针对检索目标和排除目标阈值 a. Test success：成功过滤include_similarity<0.80的目标。

- i. 创建repo2并新增一条实例: id: entity_repo2_1
repo_id: repo1 type:
feature_id value:entity1
location_id: 1
 - ii. input: 对repo1进行搜索, data是repo_id:
repo2、 value: entity_id:
entity1, type:
feature_id, 调用时search
接口,
 - iii. output: expect code 200,
entity_id is entity1,
similarity is 1, repo is
repo1 data...
2. 针对检索目标和排除目标阈值 设计
测试用例 看是否成功过滤 用例2 3暂
不测试
 - a. 是否成功过滤
include_similarity<0.80的目
标。
 - i. 流程:
 1. 删除并重建 repo-
includerthreshold。
 2. 插入实例: 三种entity
各 3 条
 - a. -entity1-
similarity: 1
 - b. -entity2-
similarity: 0.548
 - c. -entity3-
similarity: 0.921
 3. 调用 search 接口, 设置
include_threshold=0.8
0,
max_candidates=5。
 4. 检查返回结果条数: 5 -
entity1: 1-3 + entity3:
entity1-2
- b. Test error: 仍返回similarity<0.99的
entity
 - c. 001_style_api/业务测
试/search/include_threshold.py
3. 针对repositories可指定多个repoID比对
 - a. 是否成功+是否按照分数合并结果, 成
功合并, 但单个库内分数从高到低排序
输出, 但两个库结果合并只按库顺序进
行输出, 未按分数排序输出
 - i. 001_style_api/业务测
试/search/multi_repo.py
 - b. 特征长度一致不一致测试: 未实现版本
控制,前后两次不同版本的返回都成功,
且都是三个version=1.1.1的实例
 - i. 001_style_api/业务测
试/search/version_type_check.p
y
 - c. 如果两个库的level不一致, 无法进行联
合比对
4. Test error: 接口返回暂不支持
 - a. 001_style_api/业务测
试/search/muldata.py,
5. 针对repo库type和include的type不一致的
测试设计,测试成功
 - a. 001_style_api/业务测
试/search/include_threshold.py,将其
中的repo_create时的type改为face,
运行即可输出error

b. TODO是否成功过滤

similarity<0.99 的目标。

input: 设定对repo1进行
search 设定include为entity3相
关属性 设定include_threshold
为0.90 设定exclude为entity2相
关属性 设定exclude_threshold
为0.99 max_candidates=5 调用
search接口, output: (expect
result-3 of entity1: 1-3)

3. 针对repositories可指定多个repoID
比对, 设计测试用例 1.看是否成功,
是否按照分数合并结果2.repo的特征
长度需要一致 待搜索实例与repo当
中的实例的版本号一致 特征类型也
一致

□ 是否成功+是否按照分数
合并结果

i. 流程:

1. 删除并重建 repo-mul-1 与 repo-mul-2 , 确保测试从干净环境开始。
2. 等待两个仓库进入 READY 状态。
3. 每个仓库中按接口要求写入实例: 使用 body.data.value (feature) , 每个仓库新增 3 条 entity1-* (location_id="1") 和 3 条 entity2-* (location_id="0") , 以验证写入链路正常。
4. 发起联合检索请求:
POST
/repositories/search , payload 包含 include (查询特征) 、 repositories: [repo-mul-1, repo-mul-2] 、 max_candidates=12 、 topk=12 。

5. 解析检索响应并断言：

a. 使用

`extract_search_results()` 提取候选列表，确认返回非空。

b. 使用

`extract_result_repo_ids()` 验证结果同时包含两个 repo（集合等于 `{repo-mul-1, repo-mul-2}`）。

c. 使用

`extract_result_similarities()` 并断言相似度数组为非递增（`is_nonincreasing()` 为 True）。

6. 测试结束后清理：删除

`repo-mul-1` 与 `repo-mul-2`，并等待删除完成。

☐ 特征长度一致不一致测试：

i. 流程：

1. 删除并重建 repo1，写入3个 version=1.1.1 的实例。
2. include中使用 version=1.1.1的实例 发起一次 search，expect success。
3. include中再使用 version=1.1.2 发起 search，expect error。
4. 对比两次返回结果；若后端仍未强制版本校验，则保留相同结果用于回归观察。

☐ **TODO** input: 调整 entity3的type为 feature_id repo1的 当中修改实例1-5的 type为feature_id, 其中的value、 repo_id也需要设定 调用search接口搜索 repo1 (expect result entity 1-5)

☐ **TODO** input: 调整 entity3的type为 feature、 value, repo1的当中修改实例1-5的type为 feature_id 其中的 value、 repo_id也需要调整 调用search 接口搜索 repo1(expect result entity 6-20)

4. **TODO** 尝试输入两个include特征进行测试, 设计测试用例

- a. 删除并重建 repo-muldata。
- b. 插入实例: 三种entity各 3 条
 - i. -entity1-similarity: 1
 - ii. -entity2-similarity: 0.548
 - iii. -entity3-similarity: 0.921
- c. 调用 search 接口, 设置两条 data, 一个feature设置为 entity1的feature, 另一个设置为entity2的feature, include_threshold=0.99, max_candidates=5。
- d. 检查返回结果条数: 5 -entity1: 1-3 + entity2: 2 of entity1-3

5. 针对repo库type和include的type不一致的测试设计,

- a. input: 设定对repo1, repo1的 type设定为face, 进行search 设定data为feature_type为person的entity1 调用search接口, output: expect error

dete
ct

解析

1. userobject用例设计
2. TODO 感兴趣与屏蔽区域阈值测试
3. TODO 针对返回感兴趣目标在原图的结构化信息功能测试
 - a. 设定rect或rois_palygon字段 期望只针对设定区域进行检测，返回设定区域的目标

1. 做了简单测试 期望针对特定区域进行检测

rect_payload

```
1  "payload": {
2      "image": {
3          "url":
4              "http://175.168.10.27:3007
5              3/s3development/7fd35e4f-
6              f12e-46c8-b902-
7              973ab1f65498.jpg"
8      },
9      "user_object": {
10         "type": "face",
11         "rect": {
12             "x": 10,
13             "y": 10,
14             "w": 100,
15             "h": 100
16         }
17     }
18 }
```

rois_polygon_payload

```
1  "payload": {
2      "image": {
3          "url":
4              "http://175.168.10.27:3007
5              3/s3development/7fd35e4f-
6              f12e-46c8-b902-
7              973ab1f65498.jpg"
8      },
9      "rois_polygon": [
10         {
11             "polygon_area": [
12                 {
13                     "x": 0.1,
14                     "y": 0.1
15                 },
16                 {
17                     "x": 0.5,
18                     "y": 0.1
19                 }
20             ]
21         }
22     ]
23 }
```

```

15         },
16         {
17             "x": 0.5,
18             "y": 0.5
19         }
20     ],
21
22     "filter_threshold": 0.9
23 }
24 },

```

1. Expect return null test error

```

"responses": [
{
  "response_body": {
    "Result": {
      "NonMotorVehicles": [
        {
          "Img": {
            "Cutboard": {
              "Confidence": 0.91650390625,
              "Height": 188,
              "ResHeight": 0,
              "ResWidth": 0,
              "Width": 155,
              "X": 1371,
              "Y": 892
            },
            "CutboardDirection": 0,
            "CutboardSpeed": 0,
            "DetectedBox": {
              "Confidence": 0.91650390625,
              "Height": 176,
              "ResHeight": 0,
              "ResWidth": 0,
              "Width": 141,
              "X": 1378,
              "Y": 900
            },
            "Img": {
              "BinData": "",
              "Height": 188,
              "Id": "76fce9fe-9dfb-483c-82ba-2e25c459ed02",
              "Sn": 0,
              "URI": "",
              "Width": 188
            }
          }
        }
      ]
    }
  }
}
]

```